

```
In [1]: # E0: Native-language baseline
# XLM-RoBERTa fine-tuned directly on the low-resource dataset

import json
import random
import numpy as np
import pandas as pd
import torch

from torch.utils.data import Dataset, DataLoader
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    get_linear_schedule_with_warmup
)
from sklearn.metrics import (
    accuracy_score,
    f1_score,
    classification_report,
    confusion_matrix
)
```

```
/home/jay/.local/lib/python3.10/site-packages/tqdm/auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
```

```
In [2]: import sys
import torch

print("Python path:", sys.executable)
print("Torch path:", torch.__file__)
print("Torch version:", torch.__version__)
print("CUDA Available:", torch.cuda.is_available())
```

```
Python path: /usr/bin/python3
Torch path: /home/jay/.local/lib/python3.10/site-packages/torch/__init__.py
Torch version: 2.5.1+cu121
CUDA Available: True
```

```
In [3]: # =====
# 1. Configuration
# =====

MODEL_NAME = "xlm-roberta-base"

TRAIN_DATA_PATH = "db/goemotions/train.tsv"
VALID_DATA_PATH = "db/goemotions/dev.tsv"
TEST_DATA_PATH = "db/goemotions/test.tsv"
EMOTIONS_PATH = "db/goemotions/emotions.txt"
EKMAN_PATH = "db/goemotions/ekman_mapping.json"

TEXT_COLUMN = "text"
LABEL_COLUMN = "emotion"

MAX_LENGTH = 140
BATCH_SIZE = 16
EPOCHS = 10
LEARNING_RATE = 2e-5
```

```

RANDOM_SEED = 42

DEVICE = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

print("Using device:", DEVICE)

```

Using device: cuda

```

In [4]: # =====
# 2. Reproducibility
# =====

def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)

    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

set_seed(RANDOM_SEED)

```

```

In [5]: # =====
# 3. Load dataset
# =====

train_df = pd.read_csv(TRAIN_DATA_PATH, sep='\t', header=None, names=["text", "e

# Remove rows with missing text/labels
train_df = train_df.dropna(subset=[TEXT_COLUMN, LABEL_COLUMN])

print("Number of examples:", len(train_df))
print("\nClass distribution:")
print(train_df[LABEL_COLUMN].value_counts())

```

Number of examples: 43410

Class distribution:

```

emotion
27      12823
0       2710
4       1873
15      1857
1       1652
...
0,4,9,18      1
3,9,26        1
0,8,15,17     1
8,13,17       1
6,12,25       1

```

Name: count, Length: 711, dtype: int64

```

In [6]: test_df = pd.read_csv(TEST_DATA_PATH, sep='\t', header=None, names=["text", "emo

# Remove rows with missing text/labels
test_df = test_df.dropna(subset=[TEXT_COLUMN, LABEL_COLUMN])

```

```
print("Number of examples:", len(test_df))
print("\nClass distribution:")
print(test_df[LABEL_COLUMN].value_counts())
```

Number of examples: 5427

Class distribution:

emotion

27	1606
0	348
15	260
4	236
10	195

...

4,5,18	1
7,13,26	1
0,1,22	1
1,20,27	1
10,12	1

Name: count, Length: 275, dtype: int64

```
In [7]: valid_df = pd.read_csv(VALID_DATA_PATH, sep='\t', header=None, names=["text", "e

# Remove rows with missing text/labels
valid_df = valid_df.dropna(subset=[TEXT_COLUMN, LABEL_COLUMN])

print("Number of examples:", len(valid_df))
print("\nClass distribution:")
print(valid_df[LABEL_COLUMN].value_counts())
```

Number of examples: 5426

Class distribution:

emotion

27	1592
0	326
15	261
4	258
10	212

...

5,14	1
5,8,20,25	1
0,4,9	1
0,13,15,26	1
0,17,22	1

Name: count, Length: 296, dtype: int64

```
In [8]: # =====
# 4. Encode labels
# =====

with open(EMOTIONS_PATH, encoding="utf-8") as emotions_file:
    labels = [line.strip() for line in emotions_file if line.strip()]

with open(EKMAN_PATH, encoding="utf-8") as ekman_file:
    ekman_mapping = json.load(ekman_file)

label2id = {label: index for index, label in enumerate(labels)}
id2label = {index: label for label, index in label2id.items()}
```

```

category_labels = list(ekman_mapping)
category_labels.append("neutral")
category2id = {category: index for index, category in enumerate(category_labels)}
id2category = {index: category for category, index in category2id.items()}

emotion_to_category_id = {}
for category, emotions in ekman_mapping.items():
    for emotion in emotions:
        emotion_to_category_id[label2id[emotion]] = category2id[category]

emotion_to_category_id[label2id["neutral"]] = category2id["neutral"]
NUM_LABELS = len(category_labels)

def encode_category(emotion_indices):
    first_emotion_index = int(str(emotion_indices).split(",")[0])
    return emotion_to_category_id[first_emotion_index]

for df in [train_df, valid_df, test_df]:
    df["category"] = df[LABEL_COLUMN].apply(encode_category)

print("\nCategory labels:")
print(category2id)

```

Category labels:

```
{'anger': 0, 'disgust': 1, 'fear': 2, 'joy': 3, 'sadness': 4, 'surprise': 5, 'neutral': 6}
```

```

In [9]: """
# =====
# 5. Train / validation / test split
# =====

# First separate out the test set
train_val_df, test_df = train_test_split(
    df,
    test_size=0.20,
    random_state=RANDOM_SEED,
    stratify=df["category"]
)

# Then split the remaining 80% into
# 60% train and 20% validation
train_df, val_df = train_test_split(
    train_val_df,
    test_size=0.25,
    random_state=RANDOM_SEED,
    stratify=train_val_df["category"]
)
"""
print("\nDataset sizes:")
print("Train:", len(train_df))
print("Validation:", len(valid_df))
print("Test:", len(test_df))

```

Dataset sizes:

Train: 43410

Validation: 5426

Test: 5427

```

In [10]: # =====
# 6. Tokenizer

```

```
# =====  
  
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME, timeout=120)
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

```

In [11]: # =====
# 7. PyTorch Dataset
# =====

class EmotionDataset(Dataset):

    def __init__(self, dataframe, tokenizer, max_length):
        self.texts = dataframe[TEXT_COLUMN].tolist()
        self.labels = dataframe["category"].astype(int).tolist()

        self.tokenizer = tokenizer
        self.max_length = max_length

    def __len__(self):
        return len(self.texts)

    def __getitem__(self, index):

        text = str(self.texts[index])
        label = self.labels[index]

        encoding = self.tokenizer(
            text,
            truncation=True,
            padding="max_length",
            max_length=self.max_length,
            return_tensors="pt"
        )

        return {
            "input_ids": encoding["input_ids"].squeeze(0),
            "attention_mask": encoding["attention_mask"].squeeze(0),
            "labels": torch.tensor(label, dtype=torch.long)
        }

train_dataset = EmotionDataset(
    train_df,
    tokenizer,
    MAX_LENGTH
)

val_dataset = EmotionDataset(
    valid_df,
    tokenizer,
    MAX_LENGTH
)

test_dataset = EmotionDataset(
    test_df,
    tokenizer,
    MAX_LENGTH
)

```

```

In [12]: # =====
# 8. DataLoaders
# =====

train_loader = DataLoader(

```

```

    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False
)

```

```

In [13]: # =====
# 9. Model
# =====

model = AutoModelForSequenceClassification.from_pretrained(
    MODEL_NAME,
    num_labels=NUM_LABELS,
    id2label=id2category,
    label2id=category2id,
    problem_type="single_label_classification"
)

model.to(DEVICE)

```

Loading weights: 100%|██████████| 197/197 [00:00<00:00, 9782.60it/s]
[transformers] **XLNetForSequenceClassification LOAD REPORT** from: xlm-roberta-base

Key	Status
lm_head.dense.bias	UNEXPECTED
roberta.pooler.dense.bias	UNEXPECTED
roberta.pooler.dense.weight	UNEXPECTED
lm_head.layer_norm.weight	UNEXPECTED
lm_head.bias	UNEXPECTED
lm_head.layer_norm.bias	UNEXPECTED
lm_head.dense.weight	UNEXPECTED
classifier.dense.bias	MISSING
classifier.out_proj.bias	MISSING
classifier.dense.weight	MISSING
classifier.out_proj.weight	MISSING

Notes:

- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING: those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.

```

Out[13]: XLMRobertaForSequenceClassification(
  (classifier): XLMRobertaClassificationHead(
    (dense): Linear(in_features=768, out_features=768, bias=True)
    (dropout): Dropout(p=0.1, inplace=False)
    (out_proj): Linear(in_features=768, out_features=7, bias=True)
  )
  (roberta): XLMRobertaModel(
    (embeddings): XLMRobertaEmbeddings(
      (word_embeddings): Embedding(250002, 768, padding_idx=1)
      (token_type_embeddings): Embedding(1, 768)
      (LayerNorm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
      (dropout): Dropout(p=0.1, inplace=False)
      (position_embeddings): Embedding(514, 768, padding_idx=1)
    )
    (encoder): XLMRobertaEncoder(
      (layer): ModuleList(
        (0-11): 12 x XLMRobertaLayer(
          (attention): XLMRobertaAttention(
            (self): XLMRobertaSelfAttention(
              (query): Linear(in_features=768, out_features=768, bias=True)
              (key): Linear(in_features=768, out_features=768, bias=True)
              (value): Linear(in_features=768, out_features=768, bias=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
            (output): XLMRobertaSelfOutput(
              (dense): Linear(in_features=768, out_features=768, bias=True)
              (LayerNorm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
              (dropout): Dropout(p=0.1, inplace=False)
            )
          )
          (intermediate): XLMRobertaIntermediate(
            (dense): Linear(in_features=768, out_features=3072, bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): XLMRobertaOutput(
            (dense): Linear(in_features=3072, out_features=768, bias=True)
            (LayerNorm): LayerNorm((768,), eps=1e-05, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
        )
      )
    )
  )
)

```

```

In [14]: # =====
# 10. Optimizer and scheduler
# =====

optimizer = torch.optim.AdamW(
    model.parameters(),
    lr=LEARNING_RATE
)

total_training_steps = (
    len(train_loader) * EPOCHS
)

scheduler = get_linear_schedule_with_warmup(

```

```
optimizer,
num_warmup_steps=int(0.1 * total_training_steps),
num_training_steps=total_training_steps
)
```

```
In [15]: # =====
# 11. Evaluation function
# =====

def evaluate(model, dataloader):

    model.eval()

    all_predictions = []
    all_labels = []

    total_loss = 0

    with torch.no_grad():

        for batch in dataloader:

            input_ids = batch["input_ids"].to(DEVICE)
            attention_mask = batch["attention_mask"].to(DEVICE)
            labels_batch = batch["labels"].to(DEVICE)

            outputs = model(
                input_ids=input_ids,
                attention_mask=attention_mask,
                labels=labels_batch
            )

            total_loss += outputs.loss.item()
            predictions = outputs.logits.argmax(dim=-1)

            all_predictions.extend(predictions.cpu().tolist())
            all_labels.extend(labels_batch.cpu().tolist())

    average_loss = total_loss / len(dataloader)

    accuracy = accuracy_score(
        all_labels,
        all_predictions
    )

    macro_f1 = f1_score(
        all_labels,
        all_predictions,
        average="macro",
        zero_division=0
    )

    weighted_f1 = f1_score(
        all_labels,
        all_predictions,
        average="weighted",
        zero_division=0
    )

    return {
```

```

        "loss": average_loss,
        "accuracy": accuracy,
        "macro_f1": macro_f1,
        "weighted_f1": weighted_f1,
        "labels": all_labels,
        "predictions": all_predictions
    }

```

```

In [16]: # =====
# 12. Training
# =====

best_val_f1 = -float("inf")
best_model_path = "best_e0_model.pt"

for epoch in range(EPOCHS):

    model.train()

    total_train_loss = 0

    for batch in train_loader:

        optimizer.zero_grad()

        input_ids = batch["input_ids"].to(DEVICE)
        attention_mask = batch["attention_mask"].to(DEVICE)
        labels_batch = batch["labels"].to(DEVICE)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            labels=labels_batch
        )

        loss = outputs.loss

        loss.backward()

        # Prevent excessively large gradients
        torch.nn.utils.clip_grad_norm_(
            model.parameters(),
            max_norm=1.0
        )

        optimizer.step()
        scheduler.step()

        total_train_loss += loss.item()

    average_train_loss = (
        total_train_loss / len(train_loader)
    )

    # Validation
    val_results = evaluate(
        model,
        val_loader
    )

```

```

print(
    f"\nEpoch {epoch + 1}/{EPOCHS}"
)

print(
    f"Train loss: "
    f"{average_train_loss:.4f}"
)

print(
    f"Validation loss: "
    f"{val_results['loss']:.4f}"
)

print(
    f"Validation accuracy: "
    f"{val_results['accuracy']:.4f}"
)

print(
    f"Validation macro-F1: "
    f"{val_results['macro_f1']:.4f}"
)

print(
    f"Validation weighted-F1: "
    f"{val_results['weighted_f1']:.4f}"
)

# Save best model based on macro-F1
if val_results["macro_f1"] > best_val_f1:

    best_val_f1 = val_results["macro_f1"]

    torch.save(
        model.state_dict(),
        best_model_path
    )

    print("Saved new best model.")

```

Epoch 1/10
Train loss: 1.1893
Validation loss: 0.9178
Validation accuracy: 0.6749
Validation macro-F1: 0.5143
Validation weighted-F1: 0.6647
Saved new best model.

Epoch 2/10
Train loss: 0.8741
Validation loss: 0.8492
Validation accuracy: 0.6867
Validation macro-F1: 0.6086
Validation weighted-F1: 0.6855
Saved new best model.

Epoch 3/10
Train loss: 0.7585
Validation loss: 0.9002
Validation accuracy: 0.6815
Validation macro-F1: 0.5994
Validation weighted-F1: 0.6720

Epoch 4/10
Train loss: 0.6491
Validation loss: 0.9279
Validation accuracy: 0.6843
Validation macro-F1: 0.6153
Validation weighted-F1: 0.6803
Saved new best model.

Epoch 5/10
Train loss: 0.5396
Validation loss: 1.0130
Validation accuracy: 0.6825
Validation macro-F1: 0.6070
Validation weighted-F1: 0.6784

Epoch 6/10
Train loss: 0.4340
Validation loss: 1.1376
Validation accuracy: 0.6710
Validation macro-F1: 0.5957
Validation weighted-F1: 0.6635

Epoch 7/10
Train loss: 0.3432
Validation loss: 1.2937
Validation accuracy: 0.6708
Validation macro-F1: 0.6048
Validation weighted-F1: 0.6662

Epoch 8/10
Train loss: 0.2714
Validation loss: 1.4844
Validation accuracy: 0.6616
Validation macro-F1: 0.5932
Validation weighted-F1: 0.6579

Epoch 9/10

```
Train loss: 0.2131
Validation loss: 1.6837
Validation accuracy: 0.6655
Validation macro-F1: 0.5902
Validation weighted-F1: 0.6623
```

```
Epoch 10/10
Train loss: 0.1778
Validation loss: 1.7903
Validation accuracy: 0.6602
Validation macro-F1: 0.5891
Validation weighted-F1: 0.6565
```

```
In [17]: # =====
# 13. Load best model
# =====

model.load_state_dict(
    torch.load(
        best_model_path,
        map_location=DEVICE
    )
)
```

/tmp/ipykernel_1397169/2246141660.py:6: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```
    torch.load(
```

```
Out[17]: <All keys matched successfully>
```

```
In [18]: # =====
# 14. Final test evaluation
# =====

test_results = evaluate(
    model,
    test_loader
)

print("\n=====")
print("FINAL E0 TEST RESULTS")
print("=====")

print(
    f"Accuracy:      "
    f"{test_results['accuracy']:.4f}"
)

print(
    f"Macro-F1:      "
```

```

        f"{test_results['macro_f1']:.4f}"
    )

    print(
        f"Weighted-F1: "
        f"{test_results['weighted_f1']:.4f}"
    )

```

```

=====
FINAL E0 TEST RESULTS
=====
Accuracy:      0.6716
Macro-F1:      0.6002
Weighted-F1:   0.6675

```

```

In [19]: # =====
# 15. Per-category results
# =====

print("\nClassification Report:")

print(
    classification_report(
        test_results["labels"],
        test_results["predictions"],
        labels=list(range(NUM_LABELS)),
        target_names=category_labels,
        digits=4,
        zero_division=0
    )
)

```

```

Classification Report:

```

	precision	recall	f1-score	support
anger	0.5093	0.5818	0.5432	703
disgust	0.4699	0.4643	0.4671	84
fear	0.6628	0.6333	0.6477	90
joy	0.7738	0.8627	0.8158	2054
sadness	0.5552	0.5552	0.5552	317
surprise	0.5678	0.5777	0.5727	573
neutral	0.6806	0.5361	0.5998	1606
accuracy			0.6716	5427
macro avg	0.6028	0.6016	0.6002	5427
weighted avg	0.6709	0.6716	0.6675	5427

```

In [20]: # =====
# 16. Confusion matrix
# =====

category_confusion_matrix = confusion_matrix(
    test_results["labels"],
    test_results["predictions"],
    labels=list(range(NUM_LABELS))
)

print("\nCategory Confusion Matrix:")
print(
    pd.DataFrame(

```

```
category_confusion_matrix,  
index=category_labels,  
columns=category_labels  
)  
)
```

Category Confusion Matrix:

	anger	disgust	fear	joy	sadness	surprise	neutral
anger	409	21	4	75	33	39	122
disgust	25	39	2	8	2	4	4
fear	6	3	57	4	11	4	5
joy	62	2	10	1772	17	32	159
sadness	32	4	2	39	176	25	39
surprise	42	5	3	95	22	331	75
neutral	227	9	8	297	56	148	861